



Nombre y Apellido:

Legajo:

Examen Parcial 1

1. A continuación se presenta la sintaxis abstracta y una semántica operacional para un lenguaje de expresiones enteras.

Sintaxis abstracta:

$e ::= x \mid n \mid \text{op } e \ e \mid \text{if } e \text{ then } e \text{ else } e \mid \text{let } x = e \text{ in } e$
 $\text{op} ::= + \mid - \mid / \mid *$

donde $n \in \mathbb{N}$ y $x \in \text{Var}$.

La semántica operacional *big-step* está dada por una relación $\Rightarrow \subseteq ((\text{Var} \times \mathbb{N}) \times \text{Exp}) \times \mathbb{N}$, donde *Exp* es el conjunto para expresiones enteras.

$$\rho \vdash n \Rightarrow n \quad (\text{NUM}) \qquad \frac{\rho \vdash e \Rightarrow 0 \quad \rho \vdash e_1 \Rightarrow v}{\rho \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 \Rightarrow v} \quad (\text{IF0})$$

$$\rho \vdash x \Rightarrow \rho(x) \quad (\text{VAR}) \qquad \frac{\rho \vdash e \Rightarrow 0 \quad v \neq 0 \quad \rho \vdash e_2 \Rightarrow v}{\rho \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 \Rightarrow v} \quad (\text{IF1})$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \vdash e_2 \Rightarrow v_2}{\rho \vdash \text{op } e_1 \ e_2 \Rightarrow \text{App}(\text{op}, v_1, v_2)} \quad (\text{OP}) \qquad \frac{\rho \vdash e \Rightarrow v \quad \rho[x/v] \vdash e' \Rightarrow v'}{\rho \vdash \text{let } x = e \text{ in } e' \Rightarrow v'} \quad (\text{LET})$$

donde $\rho[x/v](y)$ es igual a v si $x = y$ o a $\rho(y)$ si $x \neq y$ y la relación **App** aplica la operación asociada a su primer argumento al segundo y tercer argumento.

- a) Encontrar n tal que $\emptyset \vdash \text{let } x = 0 \text{ in } (\text{let } x = 2 \text{ in } x * 2) \Rightarrow n$. Justificar usando un árbol de derivación.
- b) Una alternativa para definir la regla Let, llamada *dynamic binding*, consiste en evaluar la expresión e asociada a x recién cuando se necesite en la evaluación de e' .
 Por ejemplo, la evaluación de $\text{let } x = x + y \text{ in } (\text{let } y = 2 \text{ in } x + y)$ en un entorno ρ tal que $\rho(x) = 10$ y $\rho(y) = 20$, usando *dynamic binding* da 14 en lugar de 32.
 Definir una regla de evaluación para definiciones locales que use *dynamic binding*.
- c) Probar que el lenguaje es determinista por inducción en una de las derivaciones.

2. Escribir un parser para listas de enteros de Haskell definidas por enumeración de sus elementos, usando los constructores de listas $(:)$ y $[]$, o la operación de concatenación $++$. Los siguientes son ejemplos de cadenas a parsear:

$[1, 3, 5, 7]$
 $1 : 1 : 3 : []$
 $1 : 2 : [3, 4] ++ 5 : 6 : [7]$

Donde, por ejemplo, el resultado de correr el parser sobre la cadena " $1:2:[3,4] ++ 5: 6: [7]$ ", debe ser $([1, 2, 3, 4, 5, 6, 7], [])$.